Project - High Level Design
on

IaC Provisioning for Agriculture System

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1)	Aditya Pandey	EN22CS301057
2)	Aaditya Verma	EN22CS301009
3)	Aditya Thakre	EN22CS301069
4)	Aakansha Rajani	EN22CS301012
5)	Abhijeet Patidar	EN22CS301029

Group Name: Group 11 D1

Project Number: DO-11

Industry Mentor Name:

University Mentor Name:

Academic Year: 2026

Table of Contents

S.no		Page No.
1.	Introduction	3
1.1	Scope of the document	3
1.2	Intended Audiences	4
1.3	System overview	4
2	System Design	5
2.1	Application Design	5
2.2	Process Flow	6
2.3	Information Flow	6
2.4	Component Design	7
2.5	Key Design Considerations	7
2.6	API Catalogue	8
3	Data Design	9
3.1	Data modal	9
3.2	Data Access Mechanism	10
3.3	Data Retention Policies	11
3.4	Data Migration	11
4	Interfaces	12
5	State and Session Management	13
6	Caching	14
7	Non-Function Requirements	14
7.1	Security Aspects	15
7.2	Performance Aspects	15
8	References	16

1. Introduction

The rapid advancement of cloud computing and DevOps practices has transformed the way modern applications are developed and deployed. Traditional deployment methods often involve manual configurations, limited scalability, and higher chances of downtime. To overcome these challenges, containerization, orchestration, and automated deployment pipelines are widely adopted.

The AgriTriage system is an AI-powered agriculture support platform designed to assist users by analyzing their queries and providing intelligent recommendations. The system leverages Natural Language Processing techniques using LangChain and Groq Large Language Models to generate meaningful responses.

The application is developed using FastAPI and deployed using a DevOps-driven approach. Docker is used to containerize the application, ensuring consistency across environments. Kubernetes is used for orchestration, enabling scalability, load balancing, and high availability. Jenkins is integrated as a CI/CD tool to automate the build and deployment pipeline.

This document presents the High-Level Design (HLD) of the system, focusing on overall architecture, component interaction, and deployment strategy.

1.1 Scope of the Document

This document provides a high-level overview of the system architecture and deployment model of the AgriTriage platform. It explains how different components interact and how the system is deployed using modern DevOps tools such as Docker, Kubernetes, and Jenkins.

The document covers system design, data flow, interfaces, and non-functional requirements. It does not include low-level implementation details or internal AI model training processes.

1.2 Intended Audience

This document is intended for developers, DevOps engineers, cloud architects, and academic evaluators who want to understand the design and deployment of the system. It serves as a reference for understanding how modern DevOps practices are applied in building scalable and reliable applications.

1.3 System Overview

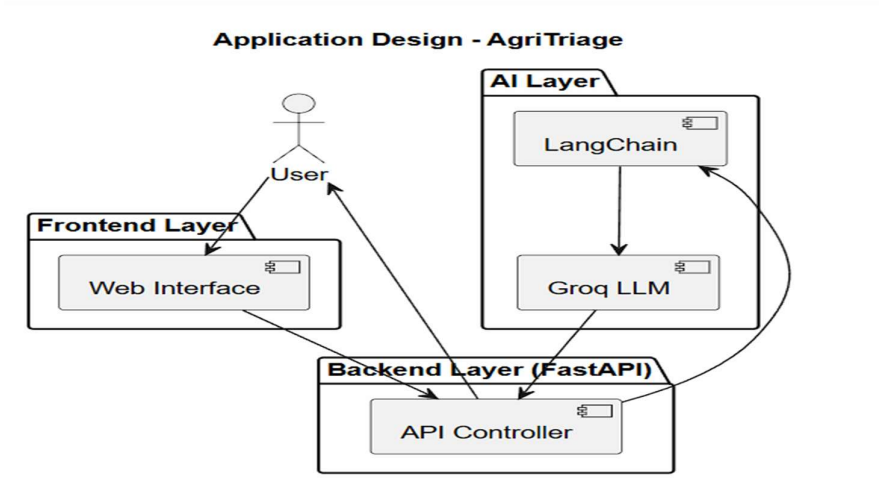
The AgriTriage system is designed as an intelligent query-processing platform that receives user inputs and generates AI-based responses. The system follows a modular architecture consisting of frontend, backend, AI processing, and deployment layers.

From a deployment perspective, the system uses Docker for containerization and Kubernetes for orchestration. Jenkins automates the CI/CD pipeline by building Docker images and deploying them to the Kubernetes cluster hosted on AWS EC2.

The system ensures scalability through multiple pods, reliability through load balancing, and efficiency through automated deployment workflows.

2. System Design

2.1 Application Design

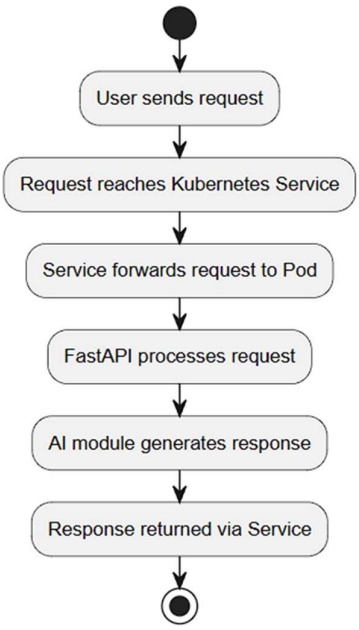


The application design follows a layered architecture consisting of frontend, backend, and AI processing layers. The frontend provides a user interface through which users submit agricultural queries. It communicates with the backend using HTTP requests in JSON format.

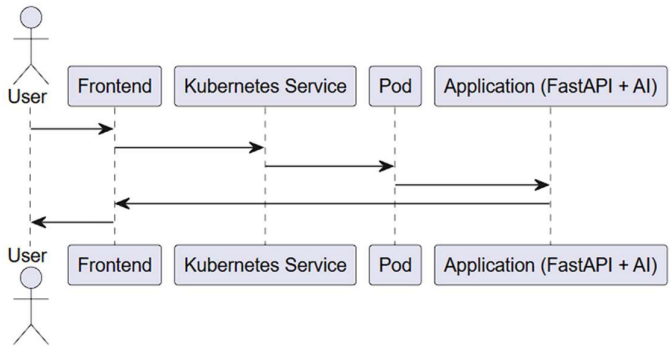
The backend is implemented using FastAPI and acts as the central controller of the system. It handles request validation, routing, and communication with the AI module. The AI layer, powered by LangChain and Groq Large Language Models, processes the query through multiple stages such as classification, entity extraction, and response generation.

This modular design ensures that each component performs a specific function, allowing the system to be easily maintained and extended. The application is containerized using Docker, which ensures consistent behavior across different environments.

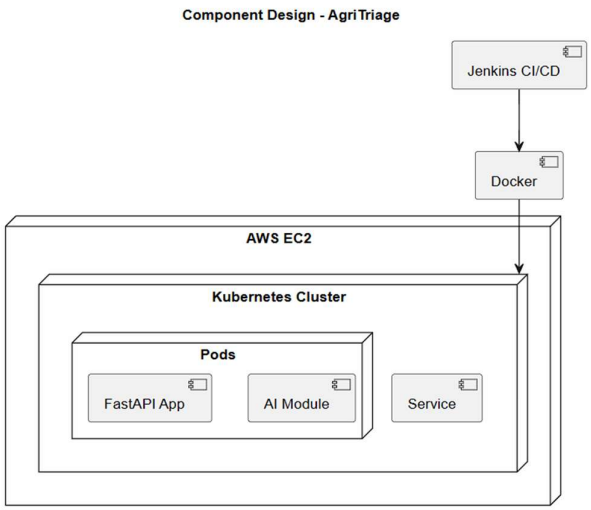
2.2 Process Flow



2.3 Information Flow



2.4 Components Design



The system is composed of several key components that work together to deliver the required functionality. The FastAPI backend is responsible for handling API requests and coordinating with the AI module. The AI module performs intelligent query processing using LangChain and Groq LLM.

Docker is used to package the application into containers, ensuring consistency across environments. Kubernetes manages the deployment and scaling of these containers by running them inside pods. A Kubernetes Service provides external access to the application and distributes traffic among pods.

Jenkins plays a critical role in automating the build and deployment process. It ensures that every code change is automatically built, tested, and deployed, reducing manual effort and improving reliability.

2.5 Key Design Considerations

The system is designed with scalability, availability, and maintainability in mind. Kubernetes enables horizontal scaling by allowing multiple pods to run simultaneously. Load balancing ensures that traffic is evenly distributed, preventing overload on any single instance.

High availability is achieved through the use of multiple pods and rolling updates, which allow new versions of the application to be deployed without downtime. Docker ensures portability by providing a consistent runtime environment.

The modular architecture improves maintainability by allowing individual components to be updated independently. Jenkins automation further enhances reliability by reducing manual errors during deployment.

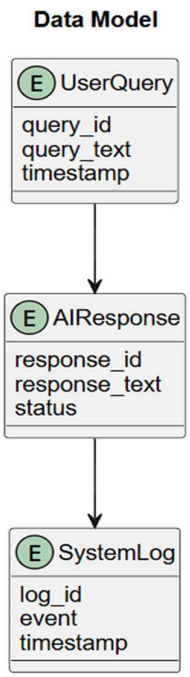
2.6 API Catalogue

The system exposes REST APIs for communication between the frontend and backend. These APIs handle query submission, processing, and system monitoring.

The APIs are designed to follow standard REST principles, ensuring simplicity and compatibility with different clients. System-level APIs are also included to monitor application health and performance.

3 Data design

3.1 Data Model



3.2 Data Access Mechanism

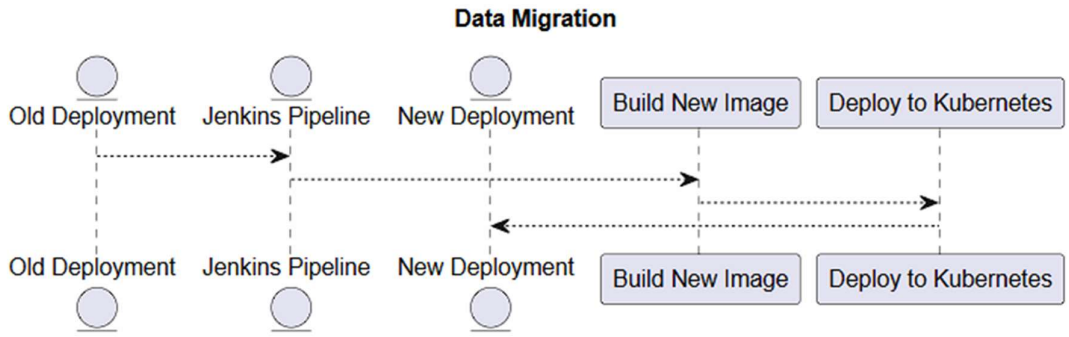
The data access mechanism is based on RESTful APIs and in-memory processing. All user interactions are handled through HTTP requests sent to the FastAPI backend. The backend processes the data and communicates with the AI module to generate responses.

The system does not rely on complex database queries. Instead, data is processed dynamically, and temporary storage mechanisms such as JSON or logs are used when necessary. This approach simplifies the architecture and improves performance.

3.3 Data Retention Policies

The system follows a minimal data retention policy, as it is primarily designed for real-time processing. User queries and responses are not stored permanently and are discarded after processing.
Temporary data may be retained for logging or debugging purposes, but it is not preserved long-term. This approach reduces storage overhead and improves system performance while also enhancing user privacy.

3.4 Data Migration



4. Interfaces

The AgriTriage system provides multiple interfaces to facilitate interaction between users and system components. The primary interface is a web-based frontend that allows users to submit agricultural queries. This interface communicates with the backend using RESTful APIs over HTTP.

The backend exposes API endpoints implemented using FastAPI, which handle request validation, processing, and response generation. These APIs serve as the communication bridge between the frontend and the AI module.

Additionally, the system interacts with external AI services through API calls to generate responses. The design ensures that all interfaces follow standardized communication protocols, making the system flexible and easy to integrate with other services.

5. State and Session Management

The AgriTriage system follows a stateless architecture, where each request is processed independently without maintaining session data on the server. This design is particularly suitable for distributed systems deployed on Kubernetes.

Incoming requests are routed by the Kubernetes Service to any available pod. Each pod processes the request independently and returns the response without relying on previous interactions. This eliminates the need for session storage and simplifies system management.

The stateless design improves scalability, as new pods can be added dynamically without affecting existing sessions. It also enhances fault tolerance, ensuring that the system continues to function even if individual pods fail.

6. Caching

Caching in the AgriTriage system is minimal, as the application primarily focuses on real-time processing of user queries. Since each query may be unique and requires dynamic AI computation, caching does not provide significant benefits in the current implementation.

Basic caching may occur at the browser level to improve user experience by reducing repeated requests. However, no dedicated caching mechanisms such as Redis or in-memory caching systems are implemented.

In future enhancements, caching strategies can be introduced to store frequently asked queries and their responses. This would help reduce processing time and improve system performance under high traffic conditions.

7. Non-Functional Requirements

7.1. Security Aspects

Security is an important aspect of the AgriTriage system and is implemented at multiple levels. At the infrastructure level, AWS security groups control access to the system by allowing only necessary ports such as SSH and application ports.

Within Kubernetes, sensitive data such as API keys and configuration parameters are managed using Kubernetes Secrets, ensuring that they are not exposed in the application code. Docker provides container-level isolation, preventing interference between different components.

At the application level, FastAPI ensures that all incoming requests are validated before processing, reducing the risk of malicious inputs.

.

7.2. Performance Aspects

The system is designed to provide efficient and scalable performance. FastAPI enables high-performance request handling due to its asynchronous capabilities, allowing the system to process multiple requests concurrently.

Kubernetes enhances performance by distributing incoming traffic across multiple pods using a load-balanced service. This prevents any single instance from becoming a bottleneck and ensures consistent response times.

Docker contributes to performance by providing lightweight containers, which consume fewer resources compared to traditional virtual machines. The system can be scaled horizontally by increasing the number of pods, allowing it to handle higher workloads efficiently.

8. References

- Amazon Web Services (AWS). *Amazon EC2 Documentation*.
Available at: https://docs.aws.amazon.com/ec2/

- Docker Inc. *Docker Official Documentation*.
Available at: https://docs.docker.com/

- FastAPI. *FastAPI Framework Documentation*.
Available at: https://fastapi.tiangolo.com/

- LangChain. *LangChain Documentation*.
Available at: https://docs.langchain.com/

- Groq Inc. *Groq API Documentation*.
Available at: https://console.groq.com/docs

- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-Based Software Architectures (REST)*. University of California.

- Sommerville, I. (2016). *Software Engineering (10th Edition)*. Pearson Education.